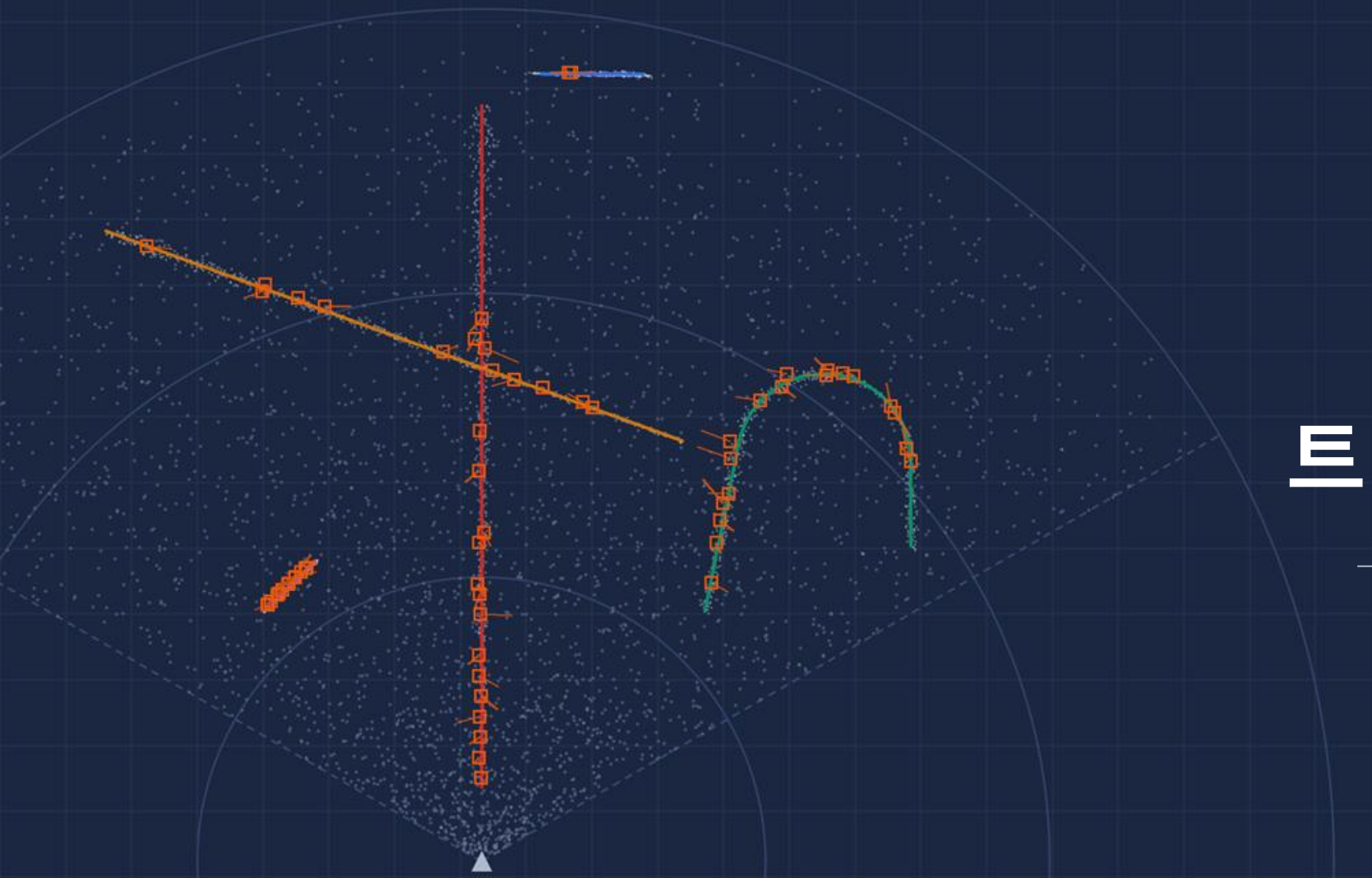
Confidential



레이다 SW 학습 과제 5

트랙 개시 · 관리

Track Initiation and Management

Janghwan Kim (Harley)

INDEX

트랙 개시 · 관리
TrackMgr

Janghwan Kim (Harley)

01 트랙 개시 3

02 트랙 관리 9

03 구현 14

04 결과 22

01. 트랙 개시

트랙이 무엇인지 보고, 플롯 두 개로 트랙을 시작하는 2 Point 개시와 속도 게이트를 설명합니다.

1.1 트랙과 트랙 개시 4

1.2 1-Point 와 2-Point 개시 5

1.3 2-Point 초기 상태변수와 오차 공분산 6

1.4 속도 게이팅 7

1.5 측정 공분산을 VC-NED 로 옮기기 8

1.1 트랙과 트랙 개시

트랙은 같은 표적의 플롯을 시간 순서로 이어 위치와 속도를 추정한 것입니다. 이번 과제는 트랙의 개시와 관리입니다.

레이다 처리 흐름에서의 위치

표적 궤적 모의
과제 3 · 시각별 참값 위치

→

측정 모의
과제 4 · 플롯 · 클러터 생성

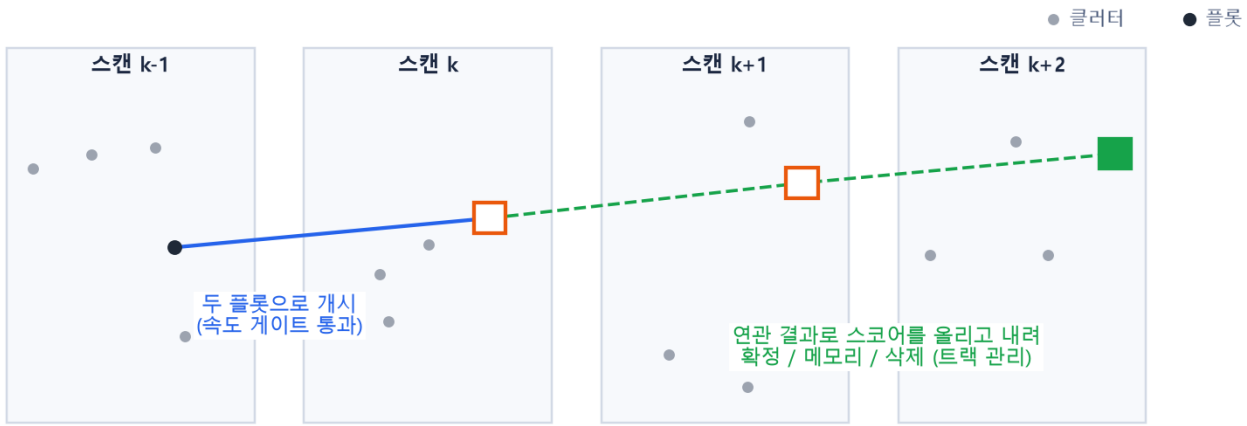
→

트랙 개시 · 관리
과제 5 · 이번 과제

→

연관 · 추적 필터
다음 과제 · 플롯을 트랙에 붙여 추정

스캔마다 들어오는 플롯과 트랙



플롯 레이다가 표적을 한 번 잰 결과. 거리 · 방위각 · 고각

클러터 표적이 아닌 것에서 나온 플롯. 같이 섞여 들어온다

트랙 개시 같은 표적으로 보이는 플롯으로 트랙을 처음 만듦

연관 이번 스캔 플롯을 기존 트랙에 이어 붙이는 것 (2.1)

트랙 관리 연관 결과로 확정 · 메모리 · 삭제를 정함

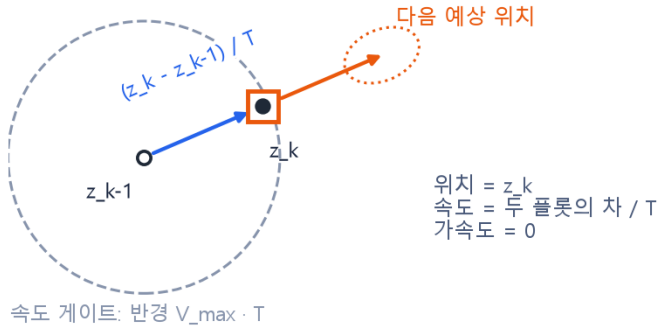
1.2 1-Point 와 2-Point 개시

플롯 하나로 바로 시작하면 속도를 모르고, 연속 두 스캔 (또는 드웰) 의 플롯으로 시작하면 위치 변화로 속도를 구할 수 있습니다.

1-Point: 플롯 하나로 개시



2-Point: 연속 두 스캔의 플롯으로 개시



1 **1-Point 개시**
장점: 플롯 하나로 바로 개시 (빠름)
단점: 속도를 몰라 초기 속도 오차가 크다

2 **2-Point 개시 (이번 과제)**
장점: 두 플롯의 위치 변화로 속도를 구해
1-Point 보다 속도 오차가 작다
단점: 연속 두 스캔 (또는 드웰) 이 필요해 늦다

3 **속도 게이트**
두 플롯이 같은 표적인지 먼저 거른다 (1.4)

1.3 2-Point 초기 상태변수와 오차 공분산

상태변수는 두 플롯으로 바로 계산하고, 오차 공분산 P 에는 그 값이 얼마나 틀릴 수 있는지를 적습니다.

1 상태변수 X 두 플롯으로 바로 계산 (세 축 x 위치 · 속도 · 가속도 = 9 개)

예 (x 축) 앞 스캔 플롯 5,000 m, 이번 스캔 플롯 5,030 m, 스캔 간격 T = 0.1 s

성분	식	예 (x 축)
위치 x	z_k	5,030 m
속도 x'	(z_k − z_k-1) / T	(5,030 − 5,000) / 0.1 = 300 m/s
가속도 x''	0 (과제 조건)	0

2 오차 공분산 P 한 축의 3 x 3 (예: 플롯 오차 ±10 m, T = 0.1 s)

	위치	속도	가속도
위치 = 이번 플롯	**R_k** 100 (±10 m)	**R_k / T** 1,000	0
속도 = 두 플롯 차 / T	**R_k / T** 1,000	**(R_k + R_k-1) / T²** 20,000 (±141 m/s)	0
가속도 = 0	0	0	0

3 9 x 9 전체 P ② 의 블록이 축마다 대각선에 하나씩

	x	x'	x''	y	y'	y''	z	z'	z''	
x	100	1,000	0	0	0	0	0	0	0	
x'	1,000	20,000	0	0	0	0	0	0	0	x 축
x''	0	0	0	0	0	0	0	0	0	
y	0	0	0	100	1,000	0	0	0	0	
y'	0	0	0	1,000	20,000	0	0	0	0	y 축
y''	0	0	0	0	0	0	0	0	0	
z	0	0	0	0	0	0	100	1,000	0	
z'	0	0	0	0	0	0	1,000	20,000	0	z 축
z''	0	0	0	0	0	0	0	0	0	

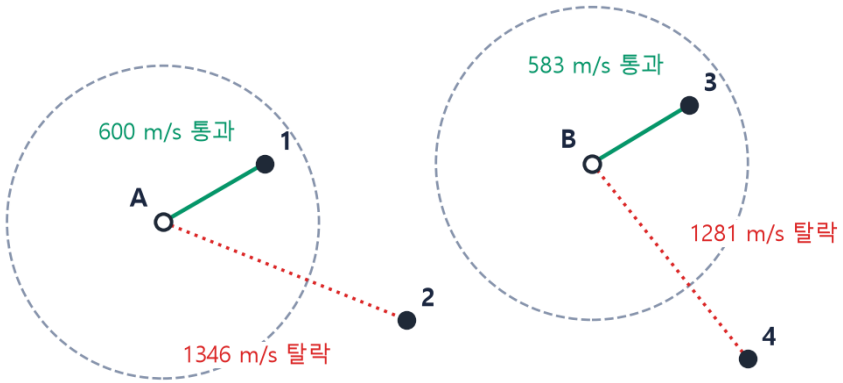
1.4 속도 게이팅

두 플롯이 같은 표적이면 한 스캔 동안 움직인 거리가 V_max · T 보다 작아야 합니다. 이 조건을 만족하는 (게이트를 통과한) 쌍으로만 개시합니다.

$$|\, z_{(k,i)} - z_{(k-1,j)} \,| \;/\; \Delta T \;<\; V_{max}$$

V_max = 800 m/s, ΔT = 0.1 s -> 앞 플롯에서 반경 80 m 안
i: 이번 스캔 플롯, j: 앞 스캔 플롯. 거리는 VC-NED 직교좌표로 잰다

짝짓기: 앞 스캔 플롯 (빈 원) 과 이번 스캔 플롯 (채운 원)



점선 원 = 반경 V_max · T = 80 m
속력이 작은 쌍부터 하나씩 고르고, 쓴 플롯은 다시 쓰지 않는다

거리별: 같은 표적 두 플롯의 차이 (파란 타원) 와 게이트 (빨간 원)



1.5 측정 공분산을 VC-NED 로 옮기기

트랙은 VC-NED (플랫폼 중심, 북 · 동 · 아래로 고정된 축) 에서 만듭니다. 거리 · 방위각 · 고각의 오차를 VC-NED 위치 오차 R 로 바꿉니다.

1 **측정 표준편차 (과제 4, SNR 20)**

$$\sigma_r = 4.74\ \text{m},\quad \sigma_{az} = 0.478°,\quad \sigma_{el} = 0.494°$$

2 **안테나 직교좌표로**

$$R_{ant} = J \cdot \mathrm{diag}(\sigma_r^2, \sigma_{az}^2, \sigma_{el}^2) \cdot J^T$$

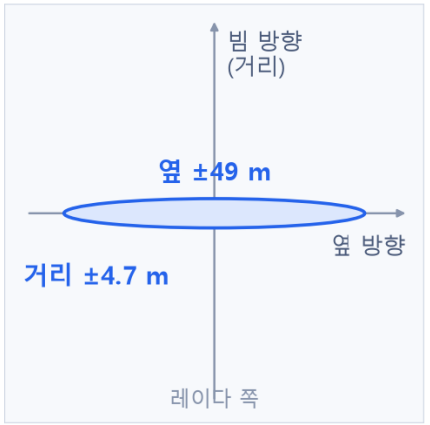
3 **VC-NED 로**

$$R = M \cdot R_{ant} \cdot M^T$$

J: 구좌표 (r, az, el) -> 안테나 직교 (x, y, z) 변환의 미분
M: 안테나 -> 동체 -> VC-NED 회전 (플랫폼 Roll · Pitch · Yaw)

표적 5 (북서쪽 5.8 km) 플롯의 오차 범위 (단위 m², 아래 방향은 생략)

빔 기준으로 보면



->

J, M

북 · 동 (VC-NED) 기준으로 보면



$$R = \begin{bmatrix} 22 & 0 \\ 0 & 2375 \end{bmatrix}$$

기울지 않아 대각선만 값이 있음

$$\begin{bmatrix} 943 & 1148 \\ 1148 & 1454 \end{bmatrix}$$

북 · 동 오차가 같이 움직여
대각선 밖 칸도 큼

02. 트랙 관리

트랙의 상태와 연관을 보고, 만든 트랙을 확정할지 지울지 정하는 M/N 방식과 트랙 스코어 방식을 설명합니다.

2.1 트랙 상태와 연관 10

2.2 M/N 방식 11

2.3 트랙 스코어 방식 12

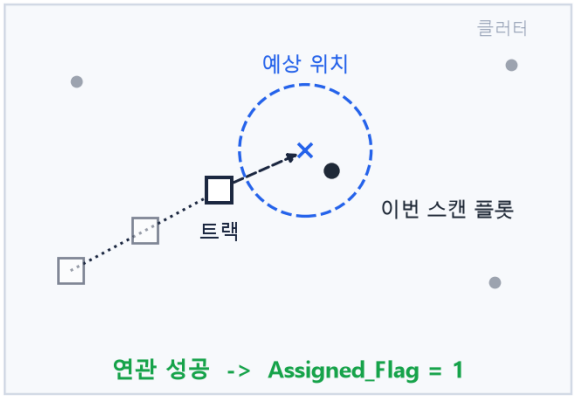
2.4 Assigned_Flag 에 따른 스코어 변화 13

2.1 트랙 상태와 연관

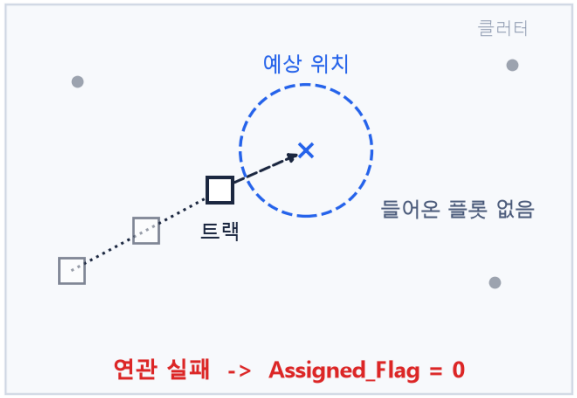
트랙은 임시로 시작해 확정, 메모리, 삭제 상태를 오갑니다. 상태를 바꾸는 근거는 스캔마다의 연관 결과 (Assigned_Flag) 입니다.

연관이란

예상 위치 근처에 이번 스캔 플롯이 있음



예상 위치 근처에 플롯이 없음



연관 이번 스캔 플롯 가운데 하나를 기존 트랙에 이어 붙이는 것
Assigned_Flag 트랙마다 스캔마다 남기는 연관 결과. 1 성공, 0 실패
이번 과제 연관은 다음 과제라 매 스캔 0 으로 둔다 (과제 조건)

트랙 상태 네 가지 TrackState

1 임시 — **막 만든 트랙. 표적인지 아직 확인하는 중**
클러터 두 개가 우연히 이어진 트랙일 수도 있다

2 확정 — **연관이 이어져 표적으로 인정한 트랙**
보통 운용자에게 알리는 것은 확정 트랙

3 메모리 — **확정 트랙이 이번 스캔에 플롯을 놓친 상태**
바로 지우지 않고 다시 잡히기를 기다린다

4 삭제 — **더 쫓지 않기로 한 트랙**
다음 스캔을 시작할 때 트랙 표에서 지운다

2.2 M/N 방식

최근 N 번의 연관 결과 가운데 성공 횟수 M 으로 확정을, 연속 실패 횟수로 삭제를 정합니다.

임시 트랙 (예: M = 3, N = 5)



N = 5 번 안에 M = 3 번 성공 -> 확정
(이 예는 스캔 4 에서 세 번째 성공)
M 번 연속 실패 -> 삭제

확정 트랙



연관 실패 -> 메모리 트랙

정한 시간보다 길게 이어지면 -> 삭제

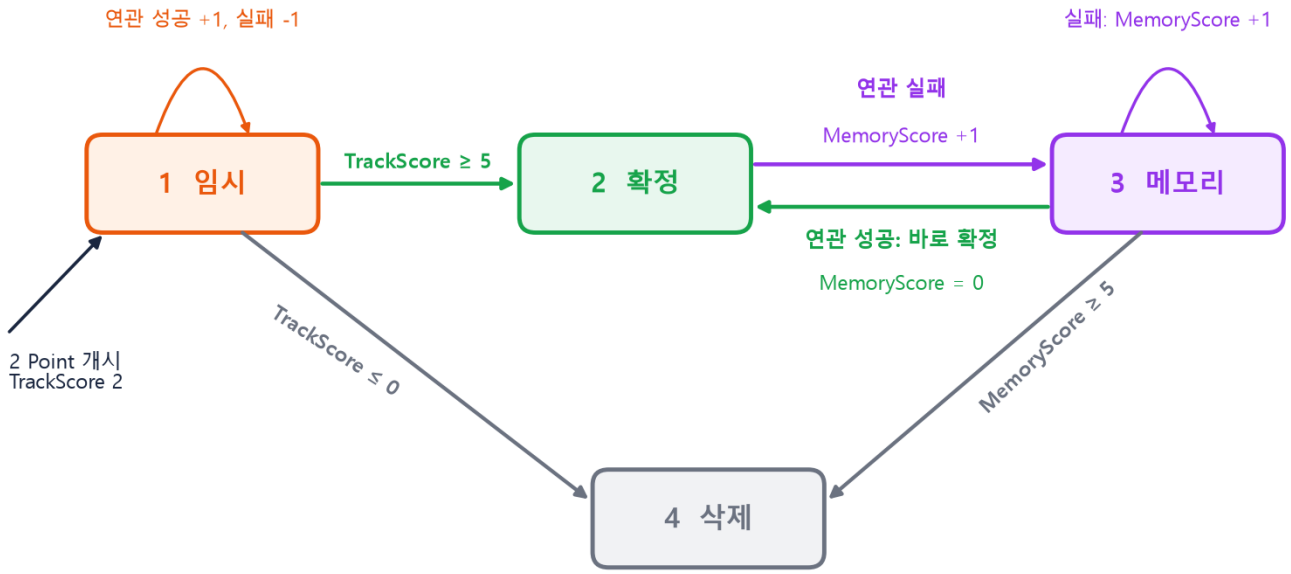
1 **임시 트랙**
N 번 중 M 번 트랙 연관에 성공하면 확정
M 번 연속 결합에 실패하면 삭제

2 **확정 트랙**
트랙 연관에 실패하면 메모리 트랙으로 바꿈
정한 시간 이상 메모리 상태면 삭제

3 **특징**
성공 · 실패 횟수만 세면 되어 단순하다
최근 N 번의 성공 · 실패 기록을 들고 있어야 한다

2.3 트랙 스코어 방식

트랙마다 TrackScore 와 MemoryScore 를 두고, 점수가 문턱을 넘으면 상태 (TrackState) 를 바꿉니다.



1. **TrackState (과제 값)**
   1 임시, 2 확정, 3 메모리, 4 삭제

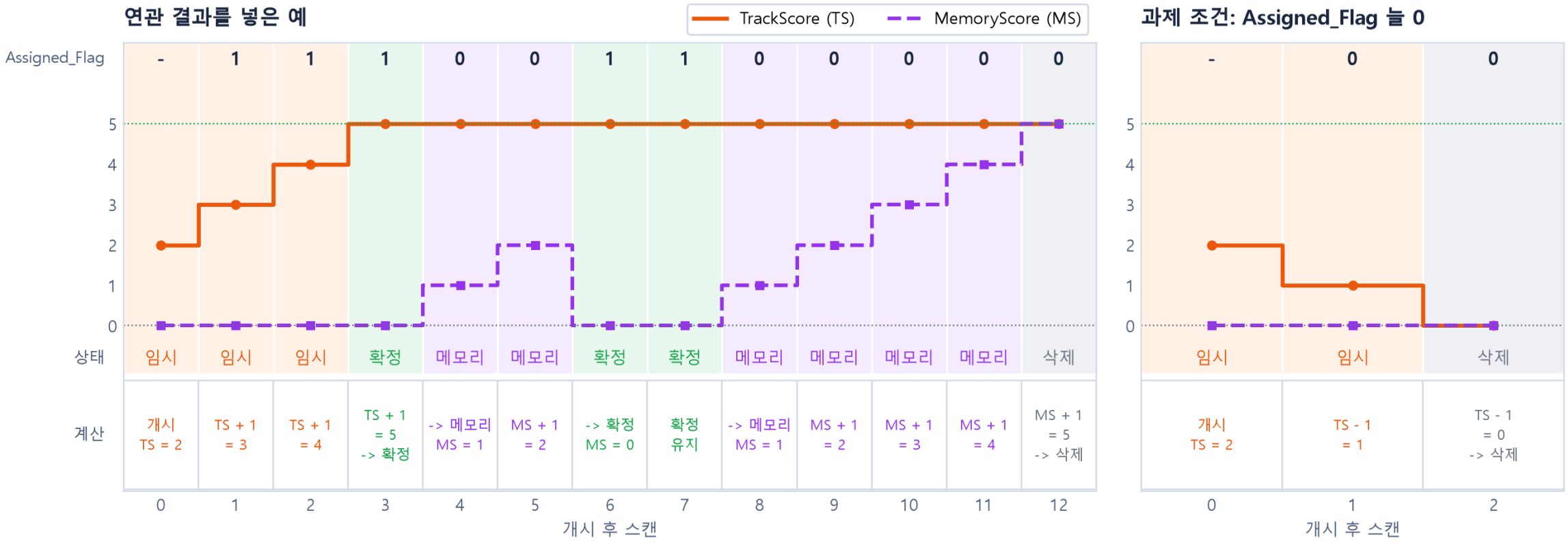
2. **문턱 (과제 값)**
   임시 트랙 확정 5, 임시 트랙 삭제 0
   메모리 트랙 삭제 5

3. **메모리 트랙이 다시 연관되면**
   바로 확정 트랙으로, MemoryScore = 0

4. **과제에 없어 정한 값**
   개시 직후 TrackScore 2 (두 플롯으로 만든 트랙)
   한 번에 올리고 내리는 양 1

2.4 Assigned_Flag 에 따른 스코어 변화

왼쪽은 연관 결과를 넣은 예, 오른쪽은 과제 조건 (매 스캔 0) 입니다. 맨 아래 계산 줄이 스캔마다 점수가 바뀐 과정입니다.



03. 구현

쓰레드 · UDP 구조와 플롯 메시지를 보고, 송신 · 수신과 트랙 개시 · 관리 코드를 차례로 봅니다.

3.1 쓰레드와 UDP 구조 15

3.2 플롯 메시지 STRUCT_PlotMsg 16

3.3 송신: 플롯 메시지 만들기 (f_Lnk_BuildMsg) 17

3.4 수신: 좌표 변환과 측정 공분산 (f_Lnk_ProcessMsg) 18

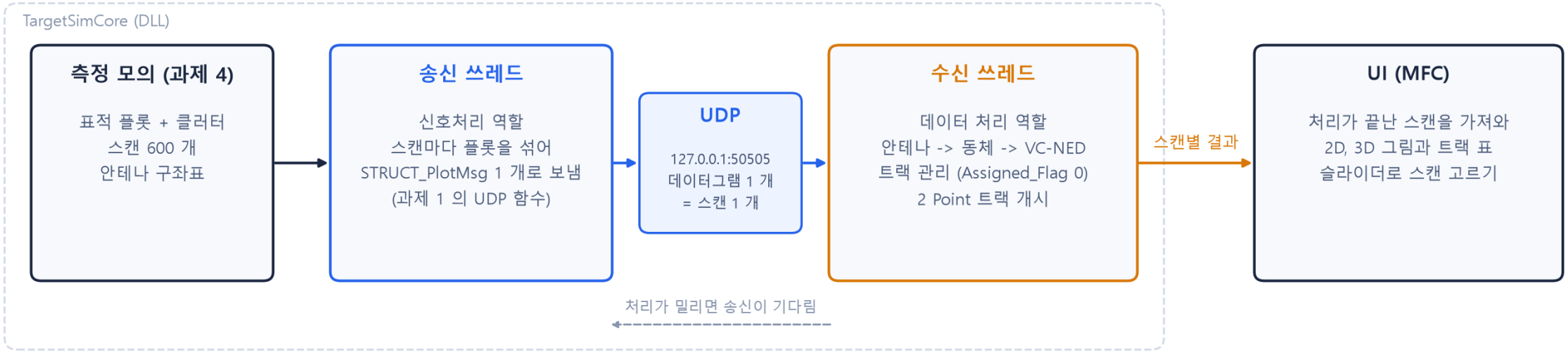
3.5 2 Point 개시: 쌍 고르기 (f_Trk_ProcessScan) 19

3.6 2 Point 초기화 (f_Trk_TwoPointInit) 20

3.7 트랙 스코어 로직 (f_Trk_UpdateState) 21

3.1 쓰레드와 UDP 구조

과제 4 의 측정 모의 결과를 신호처리가 보낸 플롯으로 보고, 송신 쓰레드가 스캔마다 UDP 로 보내면 수신 쓰레드가 받아 처리합니다.



왜 TCP 가 아니라 UDP 인가 과제는 UDP 또는 TCP/IP 중 하나를 고르게 함

	UDP (이번 과제)	TCP
메시지 단위	보낸 메시지 1 개 = 받는 메시지 1 개 (스캔 1 개)	바이트가 이어서 와서, 메시지를 직접 잘라야 함
지연	연결 · 재전송 없이 바로 전달	하나가 늦으면 뒤 스캔도 함께 기다림
손실	생길 수 있음 -> 스캔 번호로 확인 (600 / 600, 빠짐 0)	없음 (잃으면 다시 보냄)

3.2 플롯 메시지 STRUCT_PlotMsg

TargetSimCore/include/Idd.h

참고 소스 코드의 Idd.h 는 STRUCT_PlotInfo 의 Y_Body 에서 끊겨 있어 나머지와 메시지 구조체를 이어 썼습니다.

플롯 하나 STRUCT_PlotInfo

```
27  typedef struct STRUCT_PlotInfo
28  {
29    UINT32   Id;   // 플롯 ID
30    UINT32   EmittedDwellNum;   // 드웰 번호
31    UINT32   EmittedScanNum;   // 스캔 번호
32    UINT32   EmittedBeamNum;   // 빔 번호
33    UINT32   EmittedTimeStamp_s;   // 탐지 시각 (초)
34    UINT32   EmittedTimeStamp_ns;   // 탐지 시각 (나노초)
35    UCHAR8   BeamType;   // 빔 종류
36    FLOAT32   Range_Ant;   // 안테나 기준 거리 [m]
37    FLOAT32   Azimuth_Ant;   // 안테나 기준 방위각 [rad]
38    FLOAT32   Elevation_Ant;   // 안테나 기준 고각 [rad]
39    FLOAT32   Range_VcNed;   // VC-NED 기준 거리
40    FLOAT32   Azimuth_VcNed;   // VC-NED 기준 방위각
41    FLOAT32   Elevation_VcNed;   // VC-NED 기준 고각
42    DOUBLE64   X_Ant;   // 안테나 직교 x [m]
43    DOUBLE64   Y_Ant;   // 안테나 직교 y
44    DOUBLE64   Z_Ant;   // 안테나 직교 z
45    DOUBLE64   X_Body;   // 동체 직교 x [m]
46    DOUBLE64   Y_Body;   // 동체 직교 y
47    DOUBLE64   Z_Body;   // 동체 직교 z
48    DOUBLE64   X_VcNed;   // VC-NED 북 [m]
49    DOUBLE64   Y_VcNed;   // VC-NED 동
50    DOUBLE64   Z_VcNed;   // VC-NED 아래
51  }STRUCT_PlotInfo;
```

스캔 하나의 메시지 STRUCT_PlotMsg

```
62  typedef struct STRUCT_PlotMsg
63  {
64    UINT32   MsgId;   // 메시지 종류 (플롯 / 끝)
65    UINT32   MsgSize;   // 보내는 바이트 수
66    UINT32   ScanNum;   // 스캔 번호
67    UINT32   TimeStamp_s;   // 스캔 시각
68    UINT32   TimeStamp_ns;   // 스캔 시각 (나노초)
69    UINT32   PlotNum;   // 표적 플롯 + 클러터
70    FLOAT32   Sigma_Range;   // 측정 표준편차 [m]
71    FLOAT32   Sigma_Azimuth;   // [rad]
72    FLOAT32   Sigma_Elevation;   // [rad]
73    STRUCT_PlatformInfo   st_Platform;   // 플랫폼 위치 · 자세
74    STRUCT_PlotInfo   st_Plot[IDD_MAX_PLOT];   // 플롯 배열 (최대 64 개)
75  }STRUCT_PlotMsg;
```

1. 송신은 안테나 구좌표 세 칸만 채운다
2. Y_Body 다음부터 이어 쓴 칸 (받는 쪽이 계산해 채움)
3. 머리: 종류 · 크기 · 스캔 번호 · 시각 · 플롯 수
4. 측정 표준편차: 받는 쪽 오차 공분산 계산에 씀
5. 플랫폼 위치 · 자세, 플롯 배열 (최대 64 개)

3.3 송신: 플롯 메시지 만들기 (f_Lnk_BuildMsg)

TargetSimCore/src/PlotLink.c

표적 플롯과 클러터를 섞어 방위각 순서로 담고, 플롯마다 ID 를 매겨 안테나 구좌표만 채웁니다.

```
299 static UINT32 f_Lnk_BuildMsg(STRUCT_PlotMsg *st_Msg, INT32 nScan, UINT32 *pt_PlotId)  // 스캔 하나의 메시지 만들기
    ...
314   for (nIndex = 0; (nIndex < st_Scan->nTargetNum) && (nPlotNum < IDD_MAX_PLOT); nIndex++)  // 표적 플롯부터
315   {
316     st_Sph[nPlotNum]  = st_Scan->st_Plot[nIndex];  // 거리 · 방위각 · 고각
317     nSource[nPlotNum]  = nIndex + 1;  // 출처: 표적 번호 (송신 쪽 기록용)
    ...
321   for (nIndex = 0; (nIndex < st_Scan->nClutterNum) && (nPlotNum < IDD_MAX_PLOT); nIndex++)  // 이어서 클러터를 같은 배열에
322   {
323     st_Sph[nPlotNum]  = st_Scan->st_Clutter[nIndex];  // 클러터 플롯
324     nSource[nPlotNum]  = 0;  // 출처: 0 은 클러터
    ...
328   // 방위각 오름차순 (삽입 정렬)
    ...
346   st_Msg->MsgId  = IDD_MSG_ID_PLOT;  // 메시지 종류: 플롯
347   st_Msg->MsgSize  = msgSize;  // 보낼 바이트 수
348   st_Msg->ScanNum  = (UINT32)nScan;  // 스캔 번호
349   st_Msg->TimeStamp_s  = (UINT32)(timeNs / 1000000000LL);  // 스캔 시각 (초)
350   st_Msg->TimeStamp_ns  = (UINT32)(timeNs % 1000000000LL);  // 스캔 시각 (나노초)
351   st_Msg->PlotNum  = (UINT32)nPlotNum;  // 표적 + 클러터 수
    ...
365   for (nIndex = 0; nIndex < nPlotNum; nIndex++)  // 정렬된 순서대로 플롯마다
366   {
367     st_Info  = &st_Msg->st_Plot[nIndex];  // 메시지의 플롯 칸
368     st_Info->Id  = *pt_PlotId;  // 플롯 ID
    ...
375     st_Info->Range_Ant  = (FLOAT32)st_Sph[nOrder[nIndex]].r;  // 안테나 기준 거리
376     st_Info->Azimuth_Ant  = (FLOAT32)st_Sph[nOrder[nIndex]].az;  // 방위각
377     st_Info->Elevation_Ant  = (FLOAT32)st_Sph[nOrder[nIndex]].el;  // 고각
    ...
389     // 플롯 ID 는 10000 ~ 1000000 을 돌아가며 씀
390     *pt_PlotId = (*pt_PlotId >= IDD_PLOT_ID_MAX) ? IDD_PLOT_ID_MIN : (*pt_PlotId + 1U);  // 다음 플롯 ID
```

1 **표적 플롯 + 클러터를 한 배열에**
표적 플롯 뒤에 클러터를 이어 붙임
출처 (표적 k / 클러터) 는 송신 쪽에만 기록

2 **방위각 순서로 정렬**
빔이 지나가는 순서처럼
배열 위치로 표적을 알 수 없게

3 **메시지 머리**
스캔 번호, 시각, 플롯 수

4 **플롯 ID 와 안테나 구좌표**
거리 · 방위각 · 고각만 채움

5 **플롯 ID 10000 ~ 1000000**
1000000 다음은 10000 으로

3.4 수신: 좌표 변환과 측정 공분산 (f_Lnk_ProcessMsg)

TargetSimCore/src/PlotLink.c

받은 플롯을 VC-NED 로 바꾸고 측정 공분산을 구한 뒤, 트랙 관리와 개시를 한 번에 돌립니다.

```
593  static VOID f_Lnk_ProcessMsg(STRUCT_PlotMsg *st_Msg)  // 받은 메시지 하나 처리
     ...
595    const FLOAT64  time = (FLOAT64)st_Msg->TimeStamp_s + ((FLOAT64)st_Msg->TimeStamp_ns * 1.0e-9);  // 스캔 시각 [s]
     ...
610    st_Sigma.r  = st_Msg->Sigma_Range;  // 측정 표준편차: 거리                                  ① 1
611    st_Sigma.az  = st_Msg->Sigma_Azimuth;  // 방위각
612    st_Sigma.el  = st_Msg->Sigma_Elevation;  // 고각
613    st_Att.Roll  = st_Msg->st_Platform.Pf_Roll;  // 플랫폼 자세
614    st_Att.Pitch  = st_Msg->st_Platform.Pf_Pitch;
615    st_Att.Yaw  = st_Msg->st_Platform.Pf_Yaw;
     ...
617    for (nPlot = 0; nPlot < nPlotNum; nPlot++)  // 플롯마다
618    {
619      st_Info = &st_Msg->st_Plot[nPlot];  // 받은 플롯
620      f_Lnk_FillPlotCoord(st_Info, &st_Msg->st_Platform);  // 좌표 변환 (아래 세 줄)   ② 2
     ...
628      st_Plot[nPlot].pos[0]  = st_Info->X_VcNed;  // 북                                  ③ 3
629      st_Plot[nPlot].pos[1]  = st_Info->Y_VcNed;  // 동
630      st_Plot[nPlot].pos[2]  = st_Info->Z_VcNed;  // 아래
     ...
632      f_Trk_MeasCov(st_Plot[nPlot].cov, &st_Meas, &st_Sigma, &st_Att);  // 측정 공분산 R (1.5)
     ...
643      f_Trk_MoveOrigin(&s_Link.st_Table, shift);  // 앞 스캔 값을 이번 원점 기준으로   ④ 4
     ...
649    f_Trk_ProcessScan(&s_Link.st_Table, st_Plot, nPlotNum, time, (INT32)st_Msg->ScanNum, &st_Log);  // 트랙 관리 + 개시   ⑤ 5
```

① **머리에서 꺼냄**
시각, 측정 표준편차, 플랫폼 자세

② **안테나 -> 동체 -> VC-NED**
아래 세 줄. 비어 있던 좌표 칸도 채움

③ **트랙 개시용 플롯**
위치 VC-NED, 공분산 f_Trk_MeasCov (1.5)

④ **원점 옮기기**
플랫폼이 움직이면 앞 스캔 플롯 · 트랙을 이번 스캔 VC-NED 로 (이번 시나리오는 0)

⑤ **트랙 관리 + 2 Point 개시**
결과는 스캔 단위로 쌓아 둠

2 f_Lnk_FillPlotCoord 의 변환 세 줄

PlotLink.c

```
541  st_Ant  = f_Trans_Ant_Sph_To_XYZ(st_Info->Range_Ant, st_Info->Azimuth_Ant, st_Info->Elevation_Ant);  // 구좌표 -> 안테나 직교
542  st_Body  = f_Trans_Ant_To_Body(st_Ant.x, st_Ant.y, st_Ant.z);  // 안테나 -> 동체
543  st_Ned  = f_Trans_Body_To_Ned(st_Body.x, st_Body.y, st_Body.z, st_Platform->Pf_Roll, st_Platform->Pf_Yaw, st_Platform->Pf_Pitch);  // 동체 -> VC-NED (플랫폼 자세)
```

3.5 2 Point 개시: 쌍 고르기 (f_Trk_ProcessScan)

TargetSimCore/src/TrackMgr.c

모든 트랙을 Assigned_Flag 0 으로 판정한 뒤, 게이트를 통과한 쌍 가운데 가장 느린 쌍부터 하나씩 골라 트랙을 만듭니다.

```
313   st_Track->Assigned_Flag  = 0;  // 연관 기능이 없어 늘 실패 (과제 조건)
314   f_Trk_UpdateState(st_Track);  // 스코어로 상태 판정 (3.7)
      ...
369   while (isSearching != 0)  // 고를 쌍이 없을 때까지 반복
370   {
371     bestSpeed  = TRK_VMAX;  // V_max 에서 시작 = 속도 게이트
      ...
375     for (nPrev = 0; nPrev < st_Table->nPrevPlotNum; nPrev++)  // 앞 스캔 플롯
376     {
377       for (nCur = 0; nCur < nCurNum; nCur++)  // 이번 스캔 플롯 (모든 쌍)
378       {
379         if ((st_Table->st_PrevPlot[nPrev].isUsed == 0) && (st_Cur[nCur].isUsed == 0))  // 아직 안 쓴 플롯끼리만
380         {
381           speed = f_Trk_PairSpeed(&st_Table->st_PrevPlot[nPrev], &st_Cur[nCur], dt);  // |z_k - z_k-1| / T
382
383           if (speed < bestSpeed)  // 지금까지 가장 느린 쌍이면
384           {
385             bestSpeed  = speed;  // 그 쌍을 기억
386             bestPrev  = nPrev;
387             bestCur  = nCur;
      ...
399         st_Table->st_PrevPlot[bestPrev].isUsed  = 1;  // 고른 두 플롯은 다시 쓰지 않음
400         st_Cur[bestCur].isUsed  = 1;
      ...
402         newId = (st_Table->nTrackNum < TRK_MAX_TRACK) ? f_Trk_NewId(st_Table) : 0;  // 트랙 ID (100 ~ 10000)
      ...
412           f_Trk_TwoPointInit(st_Track, &st_Table->st_PrevPlot[bestPrev], &st_Cur[bestCur]);  // 초기 상태변수 · 공분산 (3.6)
413           st_Track->TrackId  = newId;
414           st_Track->TrackState  = TRK_STATE_TENTATIVE;  // 임시 트랙 (1)
415           st_Track->TrackScore  = TRK_SCORE_INIT;  // 개시 스코어 2
      ...
420           st_Track->PlotId[0]  = st_Table->st_PrevPlot[bestPrev].PlotId;  // 개시에 쓴 플롯 ID
421           st_Track->PlotId[1]  = st_Cur[bestCur].PlotId;
```

1. **트랙 관리 먼저**
   연관이 없어 모든 트랙 Assigned_Flag = 0
   f_Trk_UpdateState 로 판정 (3.7)

2. **속도 게이트**
   bestSpeed 를 V_max 로 시작
   -> 800 m/s 이상인 쌍은 고르지 않음

3. **쓴 플롯 표시**
   한 플롯은 한 쌍에만

4. **트랙 ID**
   100 부터 차례로, 10000 다음은 100
   쓰는 중인 ID 는 건너뜀

5. **초기화 (3.6)**
   임시 (1), TrackScore 2, 개시 플롯 ID 2 개

3.6 2 Point 초기화 (f_Trk_TwoPointInit)

TargetSimCore/src/TrackMgr.c

9 x 1 상태변수와 9 x 9 오차 공분산을 0 으로 만든 뒤, 축마다 위치 · 속도 칸만 채웁니다. 가속도 칸은 0 으로 남습니다.

```
164 VOID f_Trk_TwoPointInit(ST_Track *st_Track, const ST_TrkPlot *st_Prev, const ST_TrkPlot *st_Cur)  // 앞 플롯, 이번 플롯
165 {
166   const FLOAT64  dt = st_Cur->time - st_Prev->time;  // 스캔 간격 T [s]
167   matrix  *st_X = &st_Track->mat_EstX;  // 상태변수 X (9 x 1)
168   matrix  *st_P = &st_Track->mat_EstP;  // 오차 공분산 P (9 x 9)
169   INT32  nAxis;  // 축 a
170   INT32  nAxisB;  // 축 b
171
172   Matrix_initialize(st_X, TRK_STATE_DIM, 1);  // 9 x 1, 모두 0
173   Matrix_initialize(st_P, TRK_STATE_DIM, TRK_STATE_DIM);  // 9 x 9, 모두 0
174
175   for (nAxis = 0; nAxis < 3; nAxis++)  // 축 a: 0 북, 1 동, 2 아래
176   {
177     st_X->e[(3 * nAxis)][0]  = st_Cur->pos[nAxis];  // 위치 = 이번 플롯
178     st_X->e[(3 * nAxis) + 1][0]  = (st_Cur->pos[nAxis] - st_Prev->pos[nAxis]) / dt;  // 속도 = 위치 차 / T
179     st_X->e[(3 * nAxis) + 2][0]  = 0.0;  // 가속도 = 0
180
181     for (nAxisB = 0; nAxisB < 3; nAxisB++)  // 축 b
182     {
183       st_P->e[(3 * nAxis)][(3 * nAxisB)]  = st_Cur->cov[nAxis][nAxisB];  // 위치 · 위치 = R_k
184       st_P->e[(3 * nAxis)][(3 * nAxisB) + 1]  = st_Cur->cov[nAxis][nAxisB] / dt;  // 위치 · 속도 = R_k / T
185       st_P->e[(3 * nAxis) + 1][(3 * nAxisB)]  = st_Cur->cov[nAxis][nAxisB] / dt;  // 속도 · 위치 = R_k / T
          // 속도 · 속도 = (R_k + R_k-1) / T²
186       st_P->e[(3 * nAxis) + 1][(3 * nAxisB) + 1]  = (st_Cur->cov[nAxis][nAxisB] + st_Prev->cov[nAxis][nAxisB]) / (dt * dt);
187     }
188   }
189 }
```

1 **0 으로 채운 행렬**
참고 소스 코드의 Matrix_initialize
가속도 칸은 손대지 않아 0

2 **상태변수 (축 a 마다)**
위치 = 이번 플롯 (3a 번째 칸)
속도 = 위치 차 / 시간 차 (3a+1)
가속도 = 0 (3a+2)

3 **오차 공분산 (1.3 의 3 x 3 표)**
위치 · 위치 = R_k
위치 · 속도 = R_k / T
속도 · 속도 = (R_k + R_k-1) / T²

3.7 트랙 스코어 로직 (f_Trk_UpdateState)

TargetSimCore/src/TrackMgr.c

상태마다 case 하나씩입니다. 임시는 TrackScore 로, 확정 · 메모리는 MemoryScore 로 다음 상태를 정합니다.

임시 트랙

```
108 VOID f_Trk_UpdateState(ST_Track *st_Track)  // 스캔마다 트랙 하나씩
109 {
110   const INT32 isAssigned = (st_Track->Assigned_Flag != 0) ? 1 : 0;  // 연관 결과 1 / 0
111
112   switch (st_Track->TrackState)  // 지금 상태에 따라
113   {
114   case TRK_STATE_TENTATIVE:  // 임시 트랙
        // 성공 +1, 실패 -1                                                    ① 성공 +1, 실패 -1
115     st_Track->TrackScore = st_Track->TrackScore + ((isAssigned != 0) ? 1 : -1);
116
117     if (st_Track->TrackScore >= TRK_SCORE_CONFIRM)  // 5 이상이면            ② 5 이상 확정, 0 이하 삭제
118     {
119       st_Track->TrackState = TRK_STATE_CONFIRMED;  // 확정
120     }
121     else if (st_Track->TrackScore <= TRK_SCORE_DELETE)  // 0 이하면
122     {
123       st_Track->TrackState = TRK_STATE_DELETED;  // 삭제
124     }
125     else
126     {
127       // 임시 트랙 유지
128     }
129     break;
130
```

확정 트랙, 메모리 트랙

```
131   case TRK_STATE_CONFIRMED:  // 확정 트랙
132     if (isAssigned == 0)  // 연관 실패면
133     {
134       st_Track->TrackState  = TRK_STATE_MEMORY;  // 메모리로                ③ 확정 -> 메모리, MemoryScore +1
135       st_Track->MemoryScore  = st_Track->MemoryScore + 1;  // MemoryScore +1
136     }
137     break;
138
139   case TRK_STATE_MEMORY:  // 메모리 트랙
140     if (isAssigned != 0)  // 연관 성공이면
141     {
142       st_Track->TrackState  = TRK_STATE_CONFIRMED;  // 바로 확정             ④ 메모리 -> 바로 확정, MemoryScore 0
143       st_Track->MemoryScore  = 0;  // MemoryScore 0 으로
144     }
145     else  // 연관 실패면
146     {
147       st_Track->MemoryScore  = st_Track->MemoryScore + 1;  // MemoryScore +1
148
149       if (st_Track->MemoryScore >= TRK_MEMORY_DELETE)  // 5 이상이면          ⑤ MemoryScore 5 이상 삭제
150       {
151         st_Track->TrackState = TRK_STATE_DELETED;  // 삭제
152       }
153     }
154     break;
155
```

① 성공 +1, 실패 -1

② 5 이상 확정, 0 이하 삭제

③ 확정 -> 메모리, MemoryScore +1

④ 메모리 -> 바로 확정, MemoryScore 0

⑤ MemoryScore 5 이상 삭제

04. 결과

2D · 3D 전시, 트랙 수명과 스코어 확인, 거리에 따른 개시 결과를 봅니다.

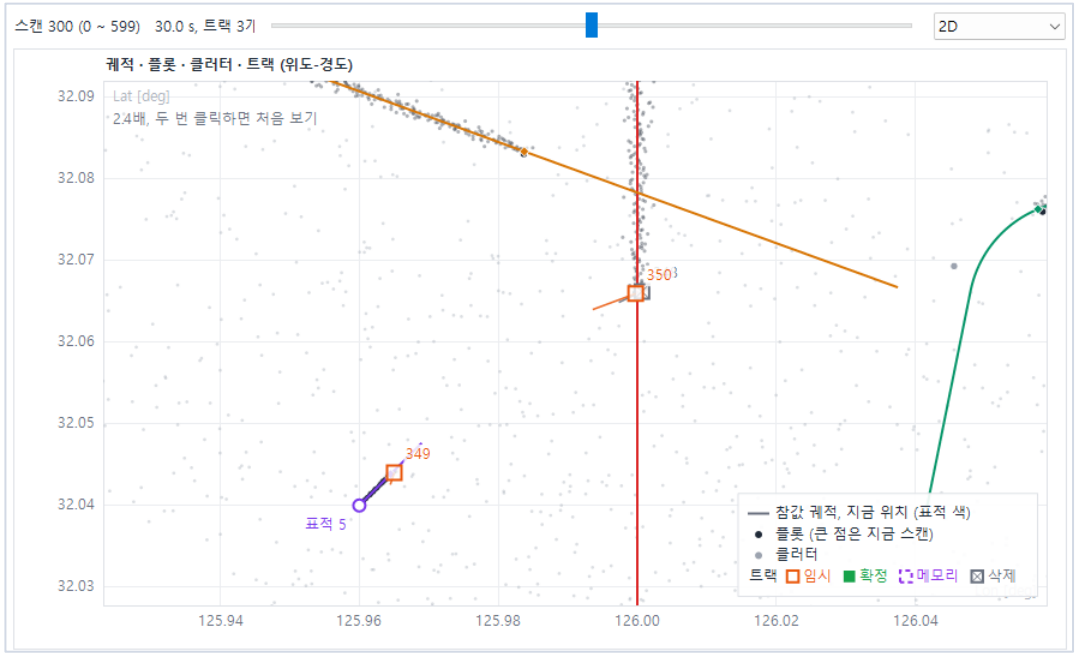
4.1 True · Plot · Track 2D · 3D 전시 23

4.2 트랙 수명과 스코어 확인 24

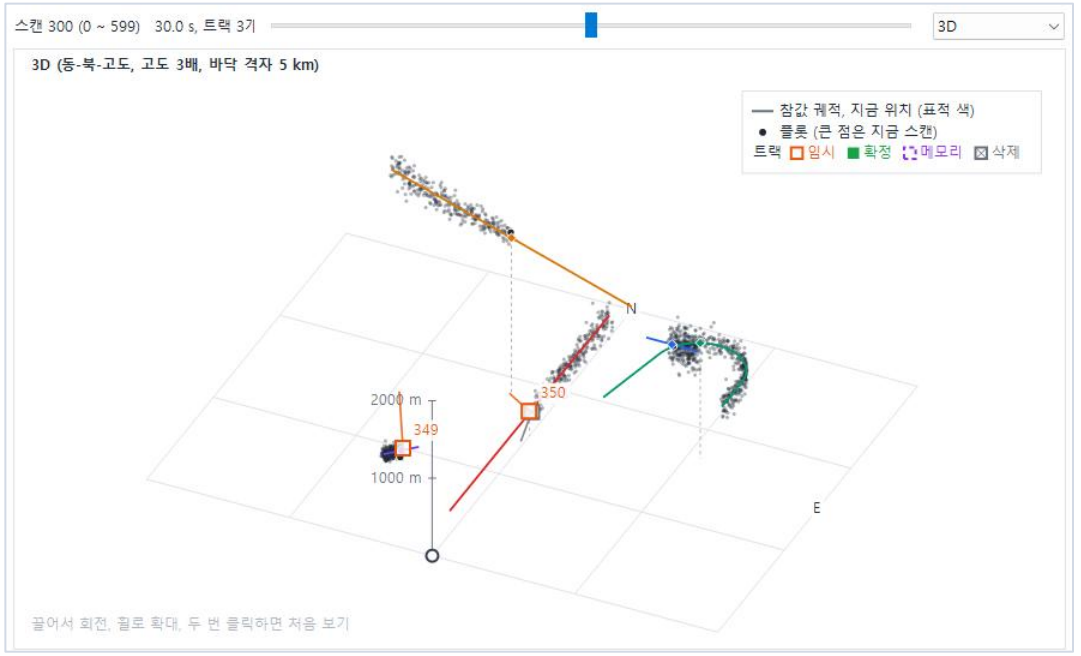
4.3 거리에 따른 개시 결과 25

4.1 True · Plot · Track 2D · 3D 전시

600 스캔에서 트랙 611 개를 개시했고 (클러터로 만든 트랙 0), 아래는 스캔 300 (30 s) 의 2D · 3D 화면입니다.



2D (위도 · 경도), 휠로 2.4 배 확대



3D, 클러터를 끄고 고도 3 배

참값 선: 궤적, 마름모: 지금 위치 (표적 색)

트랙 빈 주황 사각형: 임시, 회색 X: 삭제, 숫자: ID

플롯 옅은 점: 지난 스캔, 흰 테 큰 점: 지금 스캔

속도 선 트랙 속도로 1 초 동안 가는 거리

4.2 트랙 수명과 스코어 확인

과제 조건 (Assigned_Flag 늘 0) 에서 트랙은 세 스캔 동안만 보입니다. 확정 · 메모리 상태는 스코어 확인 창으로 확인했습니다.

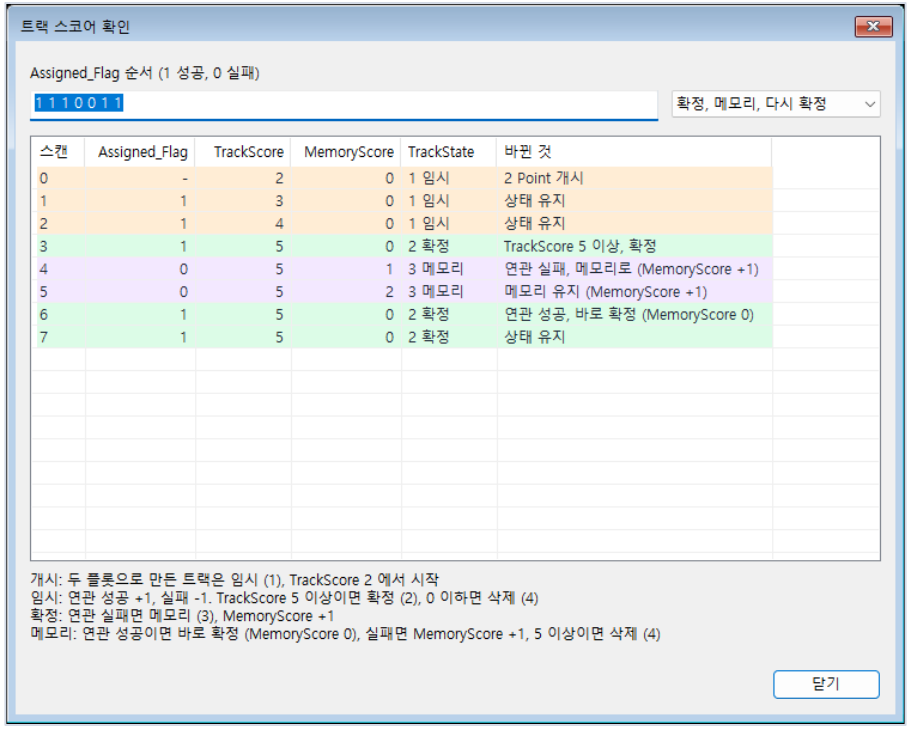
스캔 300 의 트랙 표

결과 트랙 ▾ ☐ 비중점법 비교 ☐ 플랫폼 기준 비교 CSV 저장...

트랙 ID	TrackState	TrackScore	MemoryScore	Assigned_Flag	개시 스캔	개시 플롯 (앞)	개시 플롯 (이번)	참값	북 [m]	동 [r
348	4 삭제	0	0	0	298	12974	12985	표적 2	7344.5	62
349	1 임시	1	0	0	299	12989	12999	표적 5	4881.5	-3301
350	1 임시	2	0	0	300	12997	13005	표적 2	7315.0	-23

1 **나이가 다른 트랙 세 개**
350: 이번 스캔에 개시, TrackScore 2
349: 한 스캔 전 개시, 1 / 348: 두 스캔 전 개시, 0 -> 삭제 (4)

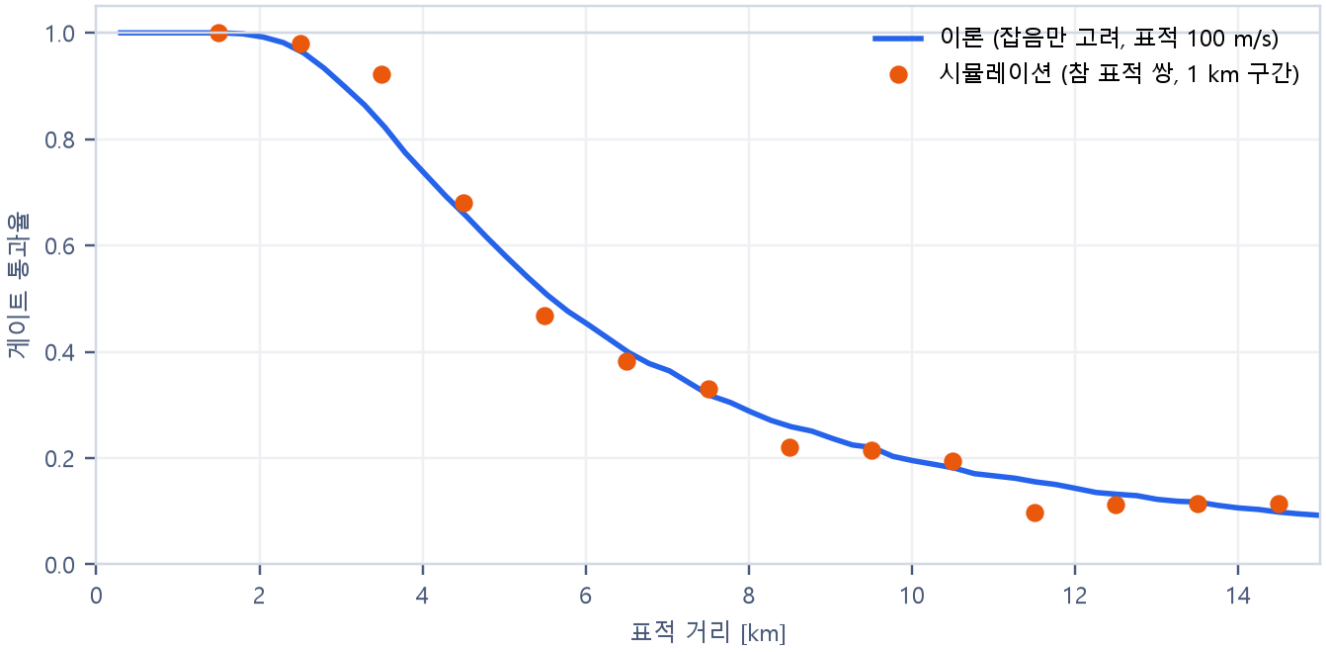
2 **모든 트랙이 같은 수명**
TrackScore 2 -> 1 -> 0 (삭제), 다음 스캔에 표에서 빠짐
611 개 모두 같은 순서



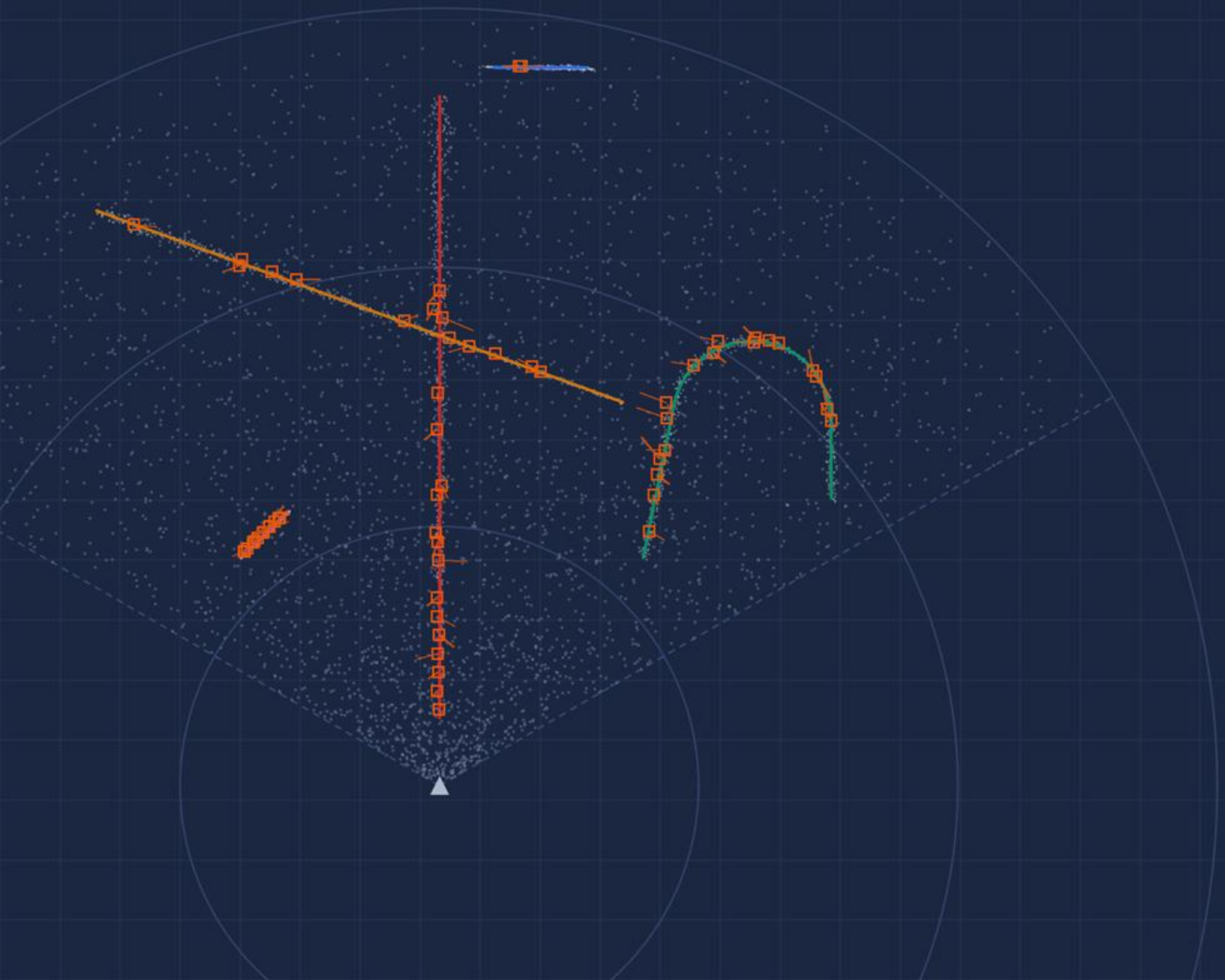
스코어 확인 창: 1 1 1 0 0 1 1 을 넣은 결과

4.3 거리에 따른 개시 결과

멀리 있는 표적일수록 같은 표적 쌍도 속도 게이트를 통과하기 어려워, 트랙이 덜 만들어집니다.



표적 (가까운 순)	거리 [km]	게이트 통과	개시 트랙
5	5.8 ~ 6.0	45 %	182
3	6.1 ~ 10.9	25 %	115
4	8.4 ~ 13.1	20 %	98
1	13.9 ~ 14.1	11 %	58
2 (다가옴)	13.3 -> 1.4	43 %	158



감사합니다

질문 주시면 답변드리겠습니다